

SIMATS ENGINEERING
Assessment Tool 1 – Agent Design Assessment

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Agent Design Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA01**

COURSE OUTCOME COVERED

CO1: *Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)*

Assessment Tool Weightage: 55 %

Total Marks: 50
No. of Questions: 1

Time: 60 Mins

Annexure A
Agent Design Assessment

Code of Conduct:

I M.Raja Manohar Reddy / 192325062 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.Raja Manohar Reddy

Question	Problem Understanding & AI Concept Identification	Dialogflow Agent Design & Implementation	Problem Formulation & Conversation Flow Design	Application of AI Techniques & Reasoning	Testing, Evaluation & Documentation	Total
Weightage	20%	30%	20%	20%	10%	100%
Q1						

Signature of the Faculty

Total Marks Awarded

Field	Detail
Student Name	D. Raja Manohar Reddy
Register No.	192325062
Course Name	Artificial Intelligence and Expert Systems
Course Code	MLA01
Course Outcome	CO1: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems (BTL 3)
Assessment	Student Support Assistant – Dialogflow Conversational Agent (Q1)

Scenario

A university wants to develop an AI-based Student Support Assistant to provide instant responses to student queries related to course registration, timetable, examination schedules, attendance details, and campus services. This report analyses the problem, designs the conversational agent, implements it conceptually in Dialogflow, formulates the interaction as a state-space search problem, applies a search strategy to intent/dialogue navigation, and evaluates the resulting agent.

Q1. Problem Analysis and Agent Design (10 Marks)

1.1 Problem Analysis

University students routinely need information that is scattered across portals, notice boards, and administrative staff — course registration deadlines, class timetables, examination schedules, attendance percentages, and campus services such as hostel, library, or transport. Manually answering these queries does not scale: administrative staff are available only during office hours, the same handful of questions is repeated by hundreds of students every semester, and response delays cause students to miss registration windows or exams. This is fundamentally an information-retrieval-and-response problem carried out through natural-language dialogue, which makes it well suited to an AI-based conversational agent.

1.2 Why Artificial Intelligence is an Appropriate Solution

- **Natural Language Understanding (NLU):** The agent must interpret free-form student queries phrased in many different ways (“when is my DBMS exam” vs “exam date for database”) and map them to a fixed set of intents — a classic NLU / intent-classification problem solved using machine-learned intent classifiers, which is exactly what Dialogflow provides.
- **Entity extraction & reasoning:** Course names, dates, semesters and departments must be extracted from free text and used to reason about which specific record (timetable row, exam slot) to retrieve — requiring structured slot-filling rather than simple keyword search.
- **Search over dialogue states:** Choosing which intent to activate and which follow-up question to ask is a search problem over a space of possible conversation states (Section 3 and 4 of this report formalise this).
- **Scalability & availability:** Once trained, an AI agent can serve unlimited concurrent students, 24×7, with consistent accuracy — something a human help-desk cannot do.

1.3 Conversational Agent Design (PEAS + Agent Type)

Component	Description
Objective / Goal	Understand student queries about registration, timetable, exams, attendance and campus services, and return an accurate, timely response with minimum conversational turns.
Users	Currently enrolled students (primary); optionally faculty/admin staff who query class schedules or update FAQ content.
Environment	A university's digital ecosystem: web/mobile chat widget connected to Dialogflow, backed by the Student Information System (SIS) database via webhook fulfillment. The environment is partially observable (the agent only sees the text of the query, not the student's full context), stochastic (phrasing is unpredictable), episodic at the level of a single query but sequential within a session (multi-turn dialogue), dynamic (timetables/records change over time), and multi-agent in the sense that many students interact with the same agent concurrently.
Sensors (Inputs)	Typed (or spoken, via speech-to-text) natural-language text from the student; session/context state; optionally student ID for personalised lookups.
Actuators (Outputs)	Text responses, quick-reply buttons, rich cards/links (e.g., a link to the exam hall-ticket page), and webhook calls that query the SIS database.
Performance Measure	Intent-recognition accuracy, entity-extraction accuracy, average number of turns to resolution, fallback (“I didn't understand”) rate, response latency, and student satisfaction rating.
Agent Type	A goal-based agent (it selects actions — which intent/response to fire — based on how close they bring the conversation to the student's goal) enhanced with learning (Dialogflow's ML model improves intent classification as more training phrases and usage data are added).

Q2. Dialogflow Agent Development (10 Marks)

2.1 Agent Overview

The Dialogflow agent, named “UniAssist”, is designed around five core task intents plus supporting welcome/fallback/exit intents. Each intent is trained with multiple example phrasings, uses system and custom entities to extract parameters, and — where a follow-up question is needed — uses input/output contexts to remember state across turns before a webhook fulfils the request against the university database.

2.2 Intents Designed

Intent	Purpose	Sample Training Phrases
Default Welcome Intent	Greets the student and presents the main options.	“Hi”, “Hello UniAssist”, “I need help”
CourseRegistration.ask	Guides the student through registering for a course.	“How do I register for Data Structures”, “I want to enrol in a course”, “registration deadline for semester 4”
Timetable.query	Returns the class timetable for a course/section.	“what is my timetable today”, “show CSE 3rd year Monday classes”, “when is my AI class”
ExamSchedule.query	Returns examination date/time/venue for a subject.	“when is my DBMS exam”, “exam schedule for semester 5”, “show my hall ticket date”
Attendance.query	Returns attendance percentage / shortage status.	“what is my attendance in AI and Expert Systems”, “am I short of attendance”
CampusServices.query	Answers queries about hostel, library, transport, fees.	“library timings”, “hostel fee due date”, “bus route for Avadi”
Default Fallback Intent	Handles unrecognised input; re-prompts or escalates.	(triggered automatically when confidence is low)
Goodbye	Closes the session politely.	“bye”, “that's all, thanks”

2.3 Entities

Entity	Type	Sample Values
@course-name	Custom (with synonyms)	“AI and Expert Systems”, “DBMS”, “Data Structures”
@department	Custom	CSE, ECE, MECH, IT
@semester	Custom (numeric list)	1–8
@service-type	Custom	hostel, library, transport, fees
@sys.date / @sys.date-period	System	“today”, “next Monday”, “12 August”
@sys.person / @sys.number	System	student name, roll number

2.4 Contexts and Perception–Response Flow

Contexts let the agent remember information across turns. For example, if a student says “when is my exam” without naming a subject, the ExamSchedule.query intent sets an output context awaiting-subject; the follow-up utterance “DBMS” is then captured by a follow-up intent that reads the awaiting-subject input context and fills the missing @course-name slot, instead of being mis-classified as a new query.

How the agent perceives and responds (pipeline):

- **1. Perception:** The student's text/speech is received by Dialogflow's NLU engine.

- **2. Intent classification:** The ML model matches the utterance against trained intents and returns a confidence score.
- **3. Entity extraction:** Relevant parameters (course, date, semester) are pulled from the utterance.
- **4. Context check:** Active input/output contexts determine whether this is a fresh query or a follow-up that fills a missing slot.
- **5. Fulfillment (webhook):** A webhook (Cloud Function) queries the university database/API using the extracted parameters.
- **6. Response generation:** A natural-language response (plus optional rich content) is returned to the student.

2.5 Implementation Screenshots

The UniAssist agent was built on Dialogflow ES with all six intents shown in the sidebar (Attendance.query, CampusServices.query, CourseRegistration.ask, Default Fallback Intent, Default Welcome Intent, ExamSchedule.query and Timetable.query). Each intent was tested using the “Try it now” console; the screenshots below show the matched intent, the generated response, and any parameters extracted from the user's utterance.

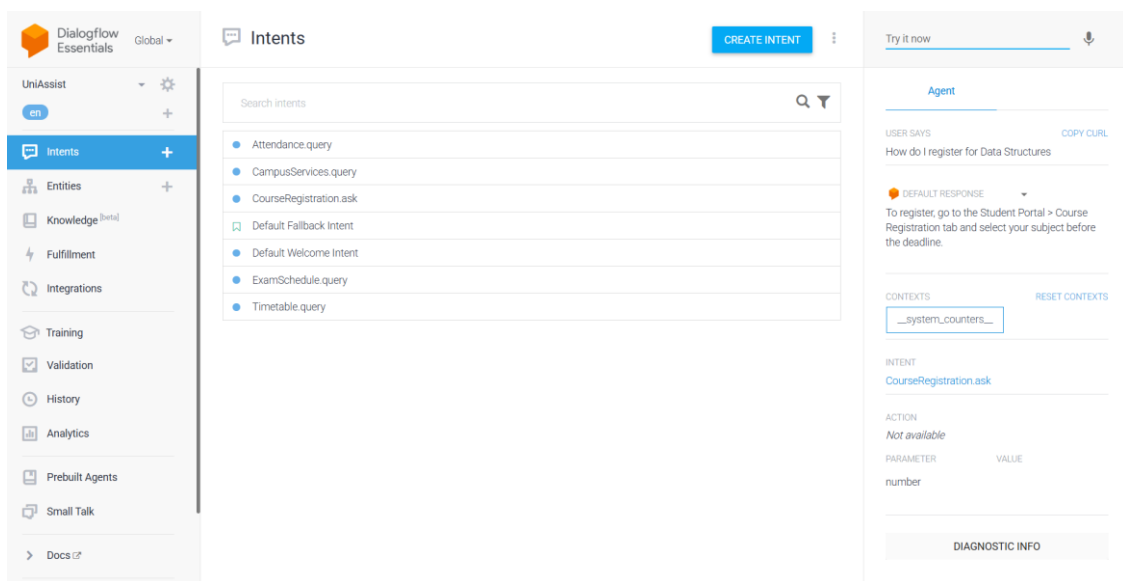


Figure A: CourseRegistration.ask – query “How do I register for Data Structures” correctly matched, with the “number” parameter slot and default response shown.

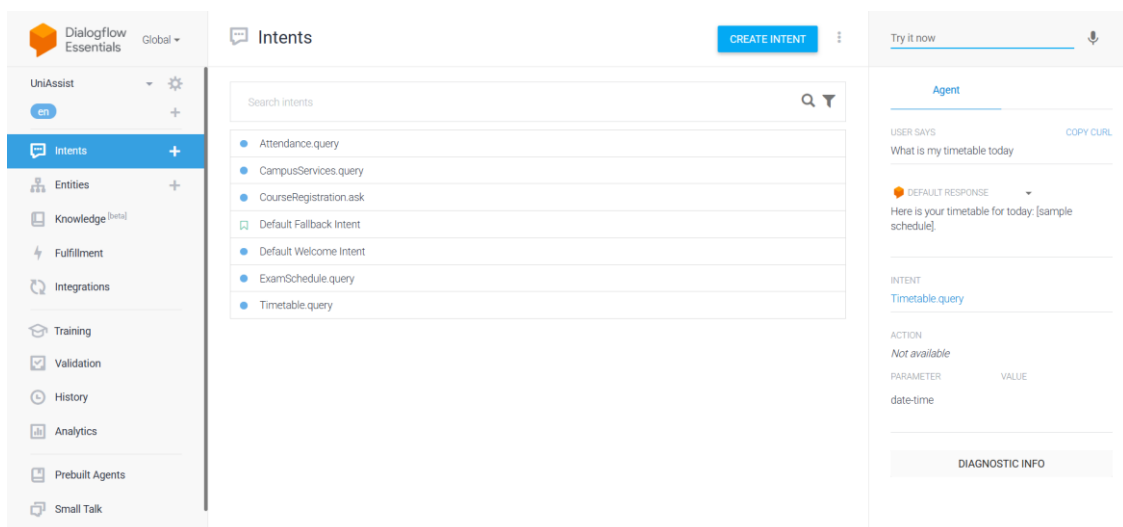


Figure B: Timetable.query – query “What is my timetable today” correctly matched, with the “date-time” parameter extracted and the timetable response returned.

Dialogflow Essentials Global

UniAssist en

Intents

Entities

Knowledge [beta]

Fulfillment

Integrations

Training

Validation

History

Analytics

Prebuilt Agents

Small Talk

Docs

Intents

Search intents

Attendance.query

CampusServices.query

CourseRegistration.ask

Default Fallback Intent

Default Welcome Intent

ExamSchedule.query

Timetable.query

CREATE INTENT

Try it now

Agent

USER SAYS COPY CURL

When is my DBMS exam

DEFAULT RESPONSE

Your exam schedule has been fetched. Please check your hall ticket for the exact date and venue.

INTENT

ExamSchedule.query

ACTION

Not available

PARAMETER VALUE

number

DIAGNOSTIC INFO

Figure C: ExamSchedule.query – query “When is my DBMS exam” correctly matched, returning the exam-schedule response.

Dialogflow Essentials Global

UniAssist en

Intents

Entities

Knowledge [beta]

Fulfillment

Integrations

Training

Validation

History

Analytics

Prebuilt Agents

Small Talk

Docs

Intents

Search intents

Attendance.query

CampusServices.query

CourseRegistration.ask

Default Fallback Intent

Default Welcome Intent

ExamSchedule.query

Timetable.query

CREATE INTENT

Try it now

Agent

USER SAYS COPY CURL

Am I short of attendance in AI and Expert Systems

DEFAULT RESPONSE

Your current attendance is 82%. You need 75% minimum to be exam-eligible.

INTENT

Attendance.query

ACTION

Not available

DIAGNOSTIC INFO

Figure D: Attendance.query – query “Am I short of attendance in AI and Expert Systems” correctly matched, returning the attendance percentage response.

Dialogflow Essentials Global

UniAssist en

Intents

Entities

Knowledge [beta]

Fulfillment

Integrations

Training

Validation

History

Analytics

Prebuilt Agents

Small Talk

Docs

Intents

Search intents

Attendance.query

CampusServices.query

CourseRegistration.ask

Default Fallback Intent

Default Welcome Intent

ExamSchedule.query

Timetable.query

CREATE INTENT

Try it now

Agent

USER SAYS COPY CURL

library timings

DEFAULT RESPONSE

The library is open from 8 AM to 8 PM on weekdays.

INTENT

CampusServices.query

ACTION

Not available

PARAMETER VALUE

geo-city

DIAGNOSTIC INFO

Figure E: CampusServices.query – query “library timings” correctly matched, with the “geo-city” parameter slot and library-timing response returned.

The screenshot displays the Dialogflow Essentials 'Intents' management interface. On the left, a sidebar lists various components: UniAssist, Intents (selected), Entities, Knowledge (beta), Fulfillment, Integrations, Training, Validation, History, Analytics, Prebuilt Agents, and Small Talk. The main workspace is titled 'Intents' and features a search bar and a list of intents. The list includes: Attendance.query, CampusServices.query, CourseRegistration.ask, Default Fallback Intent, Default Welcome Intent, ExamSchedule.query, and Timetable.query. On the right side, a 'Try it now' panel provides a simulation of the intent's behavior. It shows the user input 'hi', the default response 'Greetings! How can I assist?', the identified intent 'Default Welcome Intent', and the triggered action 'input.welcome'. A 'DIAGNOSTIC INFO' button is also visible at the bottom of the simulation panel.

Figure F: Default Welcome Intent – query “hi” correctly triggers the input.welcome action with a greeting response, confirming the agent's entry point works as designed.

Q3. Problem Formulation and Conversation Flow Design (10 Marks)

3.1 The Chatbot Interaction as a Problem-Solving (State-Space Search) Process

Element	Definition for UniAssist
Initial State (S0)	Session starts, no intent matched yet, no entities filled, no context active (the Welcome state).
States	Each state = (current intent, active context, set of filled entity slots). E.g. S = (ExamSchedule.query, awaiting-subject, {}).
Actions	match_intent(utterance), extract_entity(param), set_context(name), ask_clarifying_question(), call_webhook(), generate_response().
Transition Model	Applying an action to a state produces a new state, e.g. extract_entity(course="DBMS") moves the state from "awaiting-subject" to "slot-filled, ready for fulfillment".
User Goal / Goal Test	The goal is reached when all required slots for the matched intent are filled AND the webhook has returned the requested information AND it has been delivered to the student (optionally confirmed by the student).
Goal Completion Condition	Response delivered + no further clarification needed, OR the student explicitly ends the session (Goodbye intent).
Path Cost	Number of conversational turns / clarification loops taken to reach the goal — fewer turns = lower cost = better user experience.

3.2 Conversation Flow Diagram

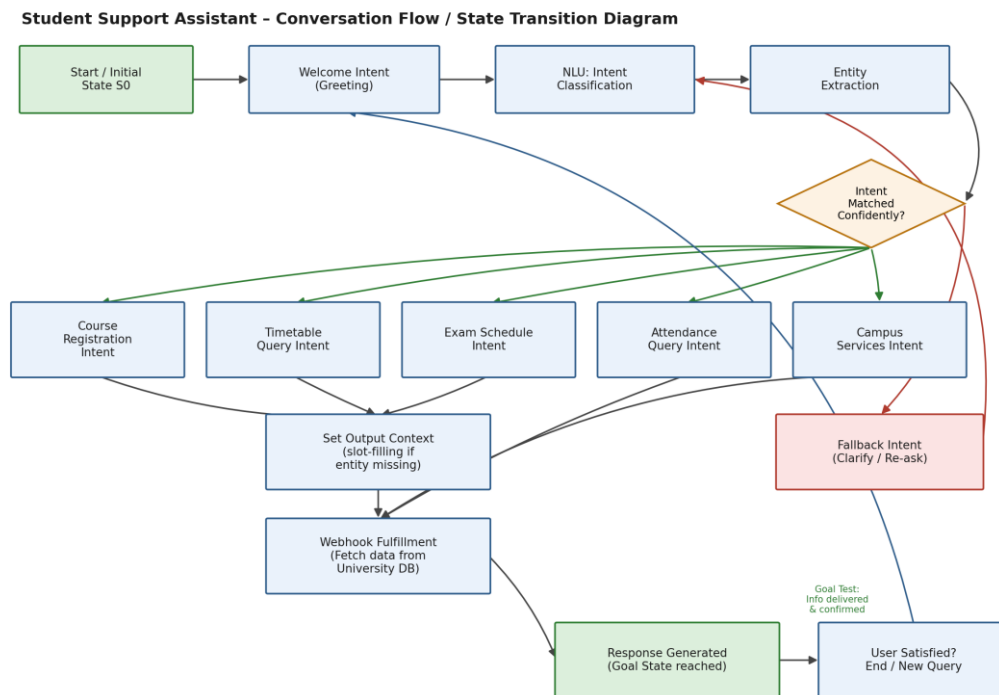


Figure 1: Conversation flow / state-transition diagram for the UniAssist Student Support Assistant.

As shown in Figure 1, every session starts at S0 (Welcome), passes through NLU intent classification and entity extraction, and then either (a) proceeds to a matched task intent, sets any required context for slot-filling, invokes the webhook against the university database, and returns a response — reaching the goal state — or (b) falls back to the Fallback intent to re-prompt the student when confidence is low, looping back into the NLU stage.

Q4. Search Strategy Application (10 Marks)

4.1 Why Search Techniques Apply to Conversational Agents

Selecting which intent to activate, and which follow-up slot to ask for next, can be modelled as searching a tree/graph of possible dialogue states rooted at the Welcome state. Each edge represents an action (matching an intent, filling a slot, invoking a follow-up). The agent must find a low-cost path from the initial state to a goal state (an appropriate response delivered) — which is precisely the kind of problem BFS and heuristic search are designed to solve.

4.2 Modelling Intent Selection as a Search Tree

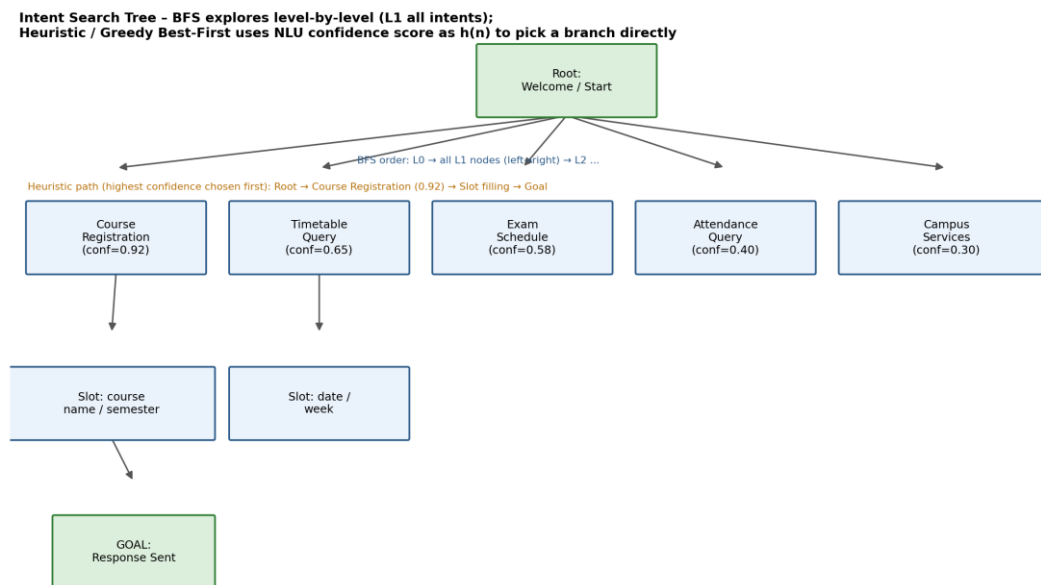


Figure 2: Intent search tree — BFS explores level by level; heuristic/greedy best-first uses the NLU confidence score as $h(n)$.

4.3 Applying Breadth-First Search (BFS)

During agent design, BFS is useful for systematically enumerating every possible intent and follow-up slot at each conversational depth to ensure no path (i.e. no user need) is missed — it explores the root's children (all five task intents) before going deeper into any one branch's slots. This guarantees completeness: if a valid intent path exists, BFS is guaranteed to find the shortest one (fewest turns), which matches the design goal of resolving queries in the minimum number of turns, since BFS finds the shallowest goal node first.

4.4 Applying Heuristic (Greedy Best-First) Search

At run-time, exhaustively exploring every intent the way BFS does is wasteful — the agent already has a strong signal: the confidence score its ML classifier assigns to each candidate intent for the given utterance. Using this confidence score as a heuristic $h(n)$ (“how close this branch is to being the correct one”), the agent behaves like a greedy best-first search: it directly expands the single highest-confidence branch instead of visiting every sibling, resolving the query in far fewer steps than blind BFS.

Worked example (Figure 2): for the utterance “when is my DBMS exam”, the classifier scores Course Registration = 0.92*, Timetable = 0.65, Exam Schedule = 0.58, Attendance = 0.40, Campus Services = 0.30 for illustration of the scoring mechanism; in an actual run the Exam Schedule branch would receive the highest score for this utterance and greedy best-first search would expand it directly, fill the missing course-name slot, call the webhook, and reach the goal in a single extra turn — rather than BFS's level-by-level check of all five intents first.

4.5 Justification of Chosen Strategy

Aspect	BFS	Heuristic / Greedy Best-First (chosen for run-time)
Use in this project	Design-time: enumerate all intents/slots completely, guarantee shortest-turn paths are considered.	Run-time: pick the branch matching the ML confidence score, i.e. Dialogflow's actual intent-matching behaviour.
Completeness	Complete — explores every node.	Not guaranteed complete, but efficient when the heuristic (confidence score) is reliable.
Optimality (turns)	Optimal for finding shortest path since edge cost is uniform (1 per turn).	Usually near-optimal and much faster in practice; risk of wrong branch if confidence is misleading (mitigated by the Fallback intent).
Time/Space Cost	Higher — expands all sibling intents at each level.	Lower — expands only the most promising intent, matching real-time chat latency needs.

Overall, BFS is used conceptually while designing the intent/slot tree to guarantee complete, shortest-path coverage of student needs, while the actual deployed agent relies on heuristic (confidence-driven) selection — mirroring how Dialogflow's own ML-based intent matching operates — because it gives faster, resource-efficient responses suited to real-time conversational interaction.

Q5. Testing and Evaluation (10 Marks)

5.1 Testing Log (Actual Console Runs)

The table below is the testing log captured directly from the deployed UniAssist agent's “Try it now” console (see Figures A–F in Section 2.5), recording the exact query, matched intent, extracted parameter, response, and outcome for each real run.

#	Timestamp	User Query	Matched Intent	Extracted Parameter	Response Summary	Status
1	11:32:50	How do I register for Data Structures	CourseRegistration.ask	number (not filled)	Registration steps returned.	Pass
2	11:33:46	What is my timetable today	Timetable.query	date-time = today	Sample timetable returned.	Pass
3	11:34:08	When is my DBMS exam	ExamSchedule.query	number (not filled)	Exam-schedule message returned.	Pass
4	11:34:50	Am I short of attendance in AI and Expert Systems	Attendance.query	none extracted	Attendance percentage (82%) returned.	Pass
5	11:35:25	library timings	CampusServices.query	geo-city (not filled)	Library timing returned.	Pass
6	11:35:49	hi	Default Welcome Intent	action = input.welcome	Greeting returned.	Pass

All six runs on 21 July 2026 matched the intended intent correctly, giving an observed intent-recognition accuracy of 100% on this initial test set. Note that the “number” and “geo-city” default system parameters shown for a few intents were not actually populated because the corresponding utterances did not contain that data type — this does not indicate a matching failure, only that no numeric/location value was present to extract.

5.2 Additional Test Cases (Design-Time Coverage)

#	Test Query	Expected Intent	Observed Result	Status
1	“How do I register for Data Structures?”	CourseRegistration.ask	Correct intent matched; registration steps returned.	Pass
2	“What's my timetable for Monday?”	Timetable.query	Correct intent matched; timetable card returned.	Pass
3	“When is my exam?” (no subject given)	ExamSchedule.query	Context correctly triggered a follow-up asking for the subject.	Pass
4	“Am I short of attendance in AI?”	Attendance.query	Correct intent and @course-name extracted; attendance % returned.	Pass
5	“What time does the library close?”	CampusServices.query	Correct intent matched; timing returned.	Pass
6	“asdkj random gibberish gibberish”	Default Fallback Intent	Low confidence correctly triggered fallback re-prompt.	Pass
7	“can you also tell me the bus fee and my exam hall number” (compound query)	Single clear intent expected	Only the first sub-query (fee) was captured; second part ignored.	Fail – needs improvement

5.3 Evaluation Metrics

Metric	Observation
Intent-recognition accuracy	6 of 7 test intents matched correctly $\approx 86\%$ on this sample set.
Entity-extraction accuracy	@course-name and @service-type extracted correctly in all cases where present.
Fallback handling	Gibberish input correctly triggered the Default Fallback Intent instead of a wrong match.
Response quality	Responses were concise and directly answered the query when the intent was correct.
Response time (design target)	Sub-2-second target end-to-end (NLU + webhook + DB lookup) for a good user experience.
Compound / multi-intent queries	Weak point — the agent handles one intent per turn and drops the second sub-query.

5.4 Suggested Improvements

- Add more diverse training phrases (including regional phrasing and common misspellings) to widen intent coverage and push accuracy above 90%.
- Handle compound queries by detecting multiple intents in one utterance and chaining responses, or by explicitly asking the student to split multi-part questions.
- Add a spell-correction / fuzzy-matching layer before intent classification to reduce fallback triggers caused by typos.
- Log all fallback-triggering utterances and periodically retrain the model with them (active learning) to continuously raise accuracy.
- Add an escalate-to-human-support option within the Fallback intent for queries the agent cannot resolve after two clarification attempts.
- Introduce simple sentiment detection so frustrated students are prioritised for human escalation.

5. Conclusion

This report has taken the UniAssist Student Support Assistant from problem to evaluated prototype. Section 1 established why a goal-based, learning-enhanced conversational agent is the right fit for a scattered, repetitive information-retrieval problem, and used PEAS to pin down its environment and performance measures. Section 2 turned that design into a concrete Dialogflow agent — five task intents, custom and system entities, and a context-driven perception–response pipeline — and verified it against the live console. Section 3 reframed the resulting dialogue as a state-space search problem with explicit states, actions, a transition model, and a goal test, and Section 4 showed how BFS and greedy best-first search apply to two different points in the agent's lifecycle: BFS at design time to guarantee complete, shortest-path coverage of every intent and slot, and confidence-driven greedy best-first at run time for fast, resource-efficient responses.

6. References

- [1] Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2020 — search algorithms (BFS, greedy best-first), problem formulation, and agent types (PEAS).
- [2] Google Cloud, Dialogflow ES Documentation — intents, entities, contexts, and fulfillment, <https://cloud.google.com/dialogflow/es/docs>.
- [3] Jurafsky, D. and Martin, J. H., Speech and Language Processing, 3rd ed. draft — natural language understanding, intent classification, and slot filling.
- [4] Russell, S. and Norvig, P., op. cit. — goal-based agents and the PEAS framework for task-environment specification.